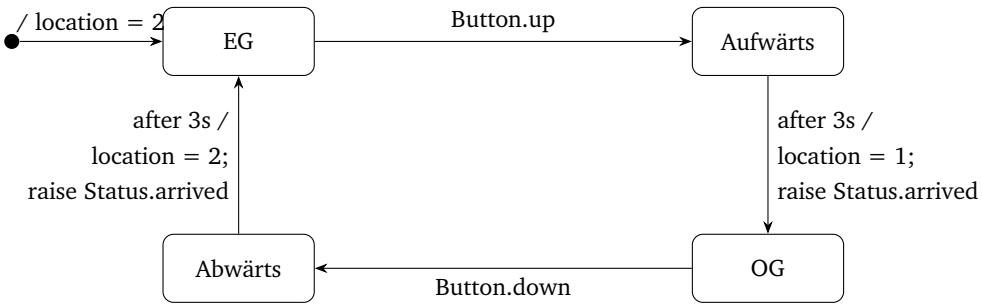
Lösungen zu Blatt 4: State Charts

Die Definitionen aus dem Aufgabenblatt werden unverändert übernommen. Alle Modelle werden mit `@EventDriven` ausgeführt. Kantenbeschriftungen verwenden `Trigger [Guard] / Effekt`; Initial- und History-Entries besitzen keinen Trigger. Nicht behandelte Eingabeereignisse werden ignoriert. Die Diagramme und Tabellen beschreiben die im CREATE-Modellbereich anzulegenden Zustände und Transitionen.

Aufgabe 1: Fahrstuhl

Vier Zustände unterscheiden Stillstand und Fahrt: *EG*, *Aufwärts*, *OG*, *Abwärts*. Die Fahrzeit wird durch Zeitevents modelliert, ohne zusätzliche Events oder Variablen.



Exakte Transitionen:

```
Initial -> EG: / location = 2
EG -> Aufwaerts: Button.up
Aufwaerts -> OG:
    after 3s / location = 1; raise Status.arrived
OG -> Abwaerts: Button.down
Abwaerts -> EG:
    after 3s / location = 2; raise Status.arrived
```

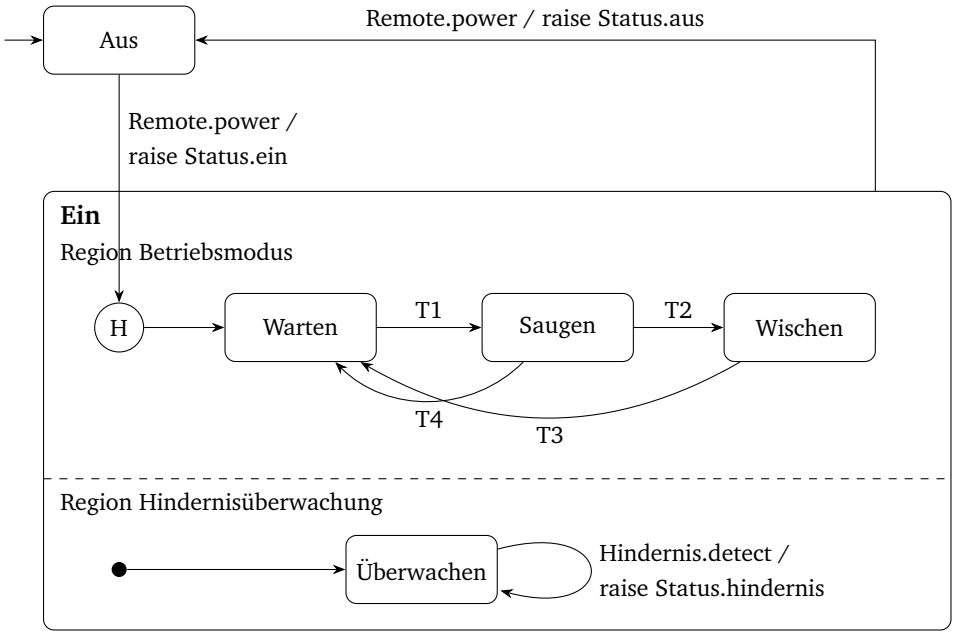
Die Schreibweise `Quelle -> Ziel:` dient hier und im Folgenden nur der Dokumentation; in CREATE wird lediglich der Text hinter dem Doppelpunkt an die jeweilige Kante geschrieben.

Während der Fahrt werden Tastenereignisse ignoriert. Insbesondere darf kein Selbstübergang in einem Fahrzustand den 3-Sekunden-Timer neu starten. `location` bleibt bis zur Ankunft das zuletzt erreichte Stockwerk; ein zusätzlicher Wert für „unterwegs“ ist nicht nötig. Beim Initialisieren wird kein Ankunftsevent ausgegeben.

Prüffolge: Start: EG, `location = 2`; `Button.up`: Aufwärts; nach 3 Sekunden: OG, `location = 1` und einmal `Status.arrived`. Die Rückfahrt verläuft symmetrisch.

Aufgabe 2a: Reinigungsroboter

Oberste Ebene: *Aus* (initial) und der zusammengesetzte Zustand *Ein*. *Ein* besitzt zwei orthogonale Regionen: *Betriebsmodus* und *Hindernisüberwachung*. Beim Einschalten wird über einen **Shallow-History-Entry** in der Modusregion der letzte Modus reaktiviert; ohne gespeicherte History führt dessen Default-Kante nach *Warten*.



Die Moduskanten sind aus Platzgründen abgekürzt; verbindlich sind folgende vollständigen Beschriftungen:

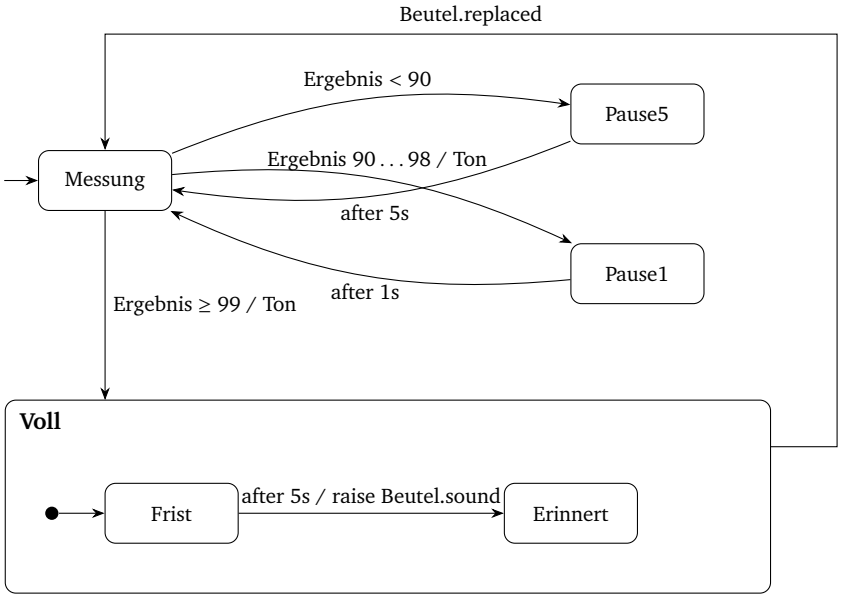
```
T1: Warten -> Saugen: Remote.mode / raise Status.saugen
T2: Saugen -> Wischen: Remote.mode / raise Status.wischen
T3: Wischen -> Warten:
    Remote.mode, Remote.stop / raise Status.warten
T4: Saugen -> Warten: Remote.stop / raise Status.warten
```

History und Hierarchie. Die Einschaltkante zielt auf den History-Entry der Modusregion in *Ein*; die zweite Region wird dabei über ihren normalen Initial-Entry betreten. Die Ausschaltkante beginnt am Rand des gesamten Zustands *Ein*. Sie beendet beide Regionen und bewahrt den letzten Modus. Shallow History reicht, da die drei Modi keine Unterzustände haben.

Ausgaben. Statusmeldungen für Moduswechsel stehen auf den Kanten, nicht als Entry-Aktionen. Dadurch wird beim Wiederherstellen nur `Status.ein` ausgegeben. Optional kann die Default-Kante $H \rightarrow Warten$ den Effekt `/ raise Status.warten` tragen, um den Modus beim allerersten Start anzuzeigen. Der Hindernis-Selbstübergang betrifft nur *Überwachen*, setzt also den Betriebsmodus nicht zurück. Im Zustand *Aus* werden Sensor-, Modus- und Stoppereignisse ignoriert.

Aufgabe 2b: Überwachung des Beutelfüllstands

Die Zustände *Messung*, *Pause5*, *Pause1* und *Voll* genügen auf oberster Ebene. *Voll* enthält die Unterzustände *Frist* und *Erinnert*. Jede Messanforderung führt nach *Messung*; dessen Entry-Aktion lautet `entry / raise Sensor.check_dust`.



Exakte Modellbeschriftungen (die Ergebniskanten im Diagramm sind abgekürzt):

```
Initial -> Messung:                    // ohne Effekt
Messung: entry / raise Sensor.check_dust
Messung -> Pause5:
    Sensor.dust_checked [Sensor.dust_level < 90]
Messung -> Pause1:
    Sensor.dust_checked
    [Sensor.dust_level >= 90 && Sensor.dust_level < 99]
    / raise Beutel.sound
Messung -> Voll:
    Sensor.dust_checked [Sensor.dust_level >= 99]
    / raise Beutel.sound
Pause5 -> Messung: after 5s
Pause1 -> Messung: after 1s
Voll.Initial -> Frist:                 // ohne Effekt
Frist -> Erinnert: after 5s / raise Beutel.sound
Voll -> Messung: Beutel.replaced
```

Warum zwei Unterzustände? Bei Eintritt in *Voll* ertönt der erste Ton. Nach 5 Sekunden folgt genau ein zweiter Ton; *Erinnert* hat keinen Timer. `Beutel.replaced` verlässt aus beiden Unterzuständen den Verbundzustand und startet sofort eine neue Messung. Beim vorzeitigen Verlassen von *Frist* wird dessen Timer beendet.

Die Guards sind disjunkt und für den zugesicherten Bereich 0–100 vollständig. Ausgewertet wird erst bei `Sensor.dust_checked`, nie direkt nach der Messanforderung. Wartezeiten beginnen mit dem verarbeiteten Ergebnis. Nebenläufigkeit ist hier nicht nötig: Messung, Pause und Warten auf Austausch erfolgen nacheinander. Bei zeitgleichem Austausch und Timerablauf ist die Reihenfolge der Ereignisverarbeitung maßgeblich; die Spezifikation legt dafür keine zusätzliche Priorität fest.